Flight Delay Lakehouse
Comprehensive Technical Project Report

Prepared for: New project member / evaluator
Scope: Ground-up technical explanation (beginner to advanced)
Generated from repository analysis of the current workspace.

1. Project Overview

What is this project?

The Flight Delay Lakehouse is a batch data engineering capstone project that builds a reproducible pipeline for large-scale U.S. aviation delay data (BTS On-Time Performance data distributed via Kaggle). It ingests raw CSV flight records, stores them in columnar Parquet files, cleans and enriches them, validates data quality, builds analytical "Gold" summary tables (marts), and produces reports/visualizations plus a lightweight data-product layer (Streamlit dashboard and a baseline ML model).

What problem does it solve?

- Raw flight data is large and not analysis-ready: mixed types, HHMM time codes, structural nulls (e.g., cancellations/diversions/cause breakdowns), and inconsistent naming.
- Analysts need reliable, fast answers (carrier performance, delay causes, routes, airport bottlenecks, hourly trends) without reprocessing raw CSV repeatedly.
- Data pipelines must be trustworthy: bad data should be detected and prevented from reaching downstream analytics.

Why was it developed?

It was developed as a Data Engineering capstone to demonstrate the end-to-end data engineering lifecycle: ingestion, lakehouse layering (Medallion), transformation/feature engineering, data quality governance, OLAP analytics, orchestration, testing, CI/CD, and documentation.

Who are its intended users?

- Course evaluators/instructors assessing data engineering competency.
- Data engineers/analysts learning lakehouse patterns on a local, manageable stack.
- Capstone team members who need a clear, reproducible pipeline and documented data model.

Main objectives

- Implement Bronze -> Silver -> Gold Medallion architecture using Parquet.
- Derive meaningful features (e.g., largest_delay_factor, flight_status, delay_severity).
- Enforce quality gates (Great Expectations framework integration when available + retained custom checks).
- Produce DuckDB analytical marts and SVG/report outputs.
- Orchestrate reliably with Apache Airflow and verify with automated tests/CI.

Simple explanation

Think of it as a factory for flight data: raw CSV goes in; it is safely stored (Bronze), cleaned and labeled (Silver), checked for quality (validation gate), summarized into business-ready tables (Gold), and finally shown as charts/dashboard/model outputs.

2. Technology Stack

Core languages, frameworks, libraries, tools

Category	Technology	Purpose / why used

Language	Python 3	Pipeline implementation, tests, orchestration callables, reporting.
Data processing	pandas, NumPy	Transformations, feature engineering, type handling, vectorized operations.
Columnar I/O	PyArrow + Apache Parquet (Snappy)	Efficient storage/compression and fast columnar reads for lakehouse layers.
Analytics/OLAP	DuckDB + SQL	In-process analytical queries over Parquet; builds Gold marts quickly.
Orchestration	Apache Airflow 2.9.3	DAG scheduling/dependencies, retries, logging, failure callbacks, blocking downstream on fai
Data quality	Great Expectations (>=0.18) + custom lightweight checks	Enterprise-style expectations when installed; retained custom checks/fallback and quality g
Visualization	Custom SVG generator (HTML/XML-based)	Reproducible, dependency-light charts from Gold marts; valid XML output.
BI / Data product	Streamlit	Interactive dashboard over Gold Parquet marts.
ML baseline	scikit-learn, joblib	Educational baseline classifier (delay > 15 min) with saved metrics/model.
Notebooks	Jupyter	Exploratory analysis (notebook present in repo).
Testing	unittest (Python stdlib)	Automated tests for ingestion/transform/validation/analytics/DAG behaviors.
CI/CD	GitHub Actions (.github/workflows/ci.yml)	Installs deps, compiles modules, runs tests; fails on errors.
Version control	Git repository	Tracks code/docs (large data often gitignored).

How technologies work together

Python scripts form each Medallion stage. pandas/PyArrow convert and transform data to/from Parquet. The validation module reads Silver Parquet and produces JSON/Markdown quality reports, halting on failure. DuckDB queries Silver Parquet via SQL to create Gold Parquet marts. The visualization module reads Gold marts and writes SVG figures. Airflow orchestrates the stage callables in order and applies reliability/failure handling. Streamlit and the ML baseline consume Gold/Silver outputs as downstream data products.

Frontend / backend / database / APIs / infrastructure

- Frontend: Streamlit dashboard (`dashboards/streamlit_app.py`) is the interactive UI layer (reads Gold marts). No separate React/Vue web frontend is implemented.
- Backend processing: Python pipeline modules under `src/` and Airflow DAG/task callables.
- Database: No traditional RDBMS/NoSQL server used as primary storage. Data is stored as files (CSV/Parquet). DuckDB is an embedded/in-process analytical engine (not a separate DB server).
- APIs: No external HTTP API endpoints implemented by this project. Airflow `notify_failure` can optionally POST JSON to `PIPELINE_ALERT_WEBHOOK` if configured.
- Infrastructure: Local filesystem + Python environment/.venv; GitHub Actions for CI; Airflow runtime components (scheduler/webserver/metadata DB) are not fully provisioned in-repo beyond the DAG and dependencies.

3. Project Architecture

Overall system architecture

The system follows the Medallion Lakehouse pattern: Bronze (raw/source-preserving), Silver (cleaned/typed/enriched), Gold (analytics-ready marts). A data quality gate sits between Silver and Gold. Orchestration is handled by an Airflow DAG with strict sequential dependencies so failures block downstream processing.

```
Raw CSV (data/raw/flights.csv)
  -> src/ingestion/ingest.py -> data/bronze/flights.parquet
  -> src/transformations/clean_flights.py -> data/silver/flights_cleaned.parquet
  -> src/validation/validate_flights.py (GX + custom; gate)
       -> reports/validation_results.json, reports/validation_report.md
  -> src/analytics/build_gold.py (DuckDB SQL over Silver)
       -> data/gold/*_summary.parquet + dim_*.parquet
  -> src/reporting/visualize.py -> reports/figures/*.svg
Downstream products: dashboards/streamlit_app.py, src/analytics/predict_delay.py
```

Component communication

- Airflow `PythonOperator` tasks call Python functions directly (in-process), passing context/logging; they share data via filesystem artifacts (Parquet/JSON/SVG), not network messages.
- DuckDB reads `data/silver/flights_cleaned.parquet` using SQL and writes Gold Parquet files.
- Validation reads Silver and writes reports; on failure it exits/raises to stop downstream stages.

Complete application workflow

- 1) Ingest: read raw CSV with selected string dtypes; write Bronze Parquet; log compression metrics.
- 2) Transform: rename to snake_case; parse HHMM timestamps (including 2400 rollover); derive route/date/hour/day, delays, status/severity, largest_delay_factor, cause flags; assign flight_id; write Silver Parquet.
- 3) Validate: run GX-native expectations if available + retained custom checks; generate reports; fail gate if any unsuccessful expectation (or simulated failure).
- 4) Build Gold: create DuckDB view over Silver; run aggregation SQL; write canonical marts and materialized dims.
- 5) Visualize: read Gold marts; generate XML-safe SVG charts with dynamic heights.

Important modules and responsibilities

Module/file	Responsibility
src/ingestion/ingest.py	CSV -> Bronze Parquet; default path resolution; compression metrics.
src/transformations/clean_flights.py	Bronze -> Silver; schema standardization, datetime parsing, feature engineering.
src/validation/validate_flights.py	Data quality suite (GX + custom), reports, blocking gate, CLI.
src/analytics/build_gold.py	DuckDB Gold marts + dim_airline/dim_airport materialization.
src/reporting/visualize.py	Gold marts -> SVG figures (escaped/dynamic layout).
airflow/dags/flight_delay_pipeline.py	Airflow DAG definition, task wrappers/logging, retries, failure callback.
run_pipeline.py	Simple sequential non-Airflow runner for local execution.
dashboards/streamlit_app.py	Interactive BI over Gold marts.
src/analytics/predict_delay.py	Baseline ML model/metrics artifact generation.

4. Folder and File Structure

Top-level structure (key paths)

Path	Purpose
airflow/dags/flight_delay_pipeline.py	Airflow DAG orchestrating the 5 pipeline tasks.
src/ingestion/	Bronze ingestion code.

src/transformations/	Silver cleaning/feature engineering.
src/validation/	Data quality validation and reporting.
src/analytics/	Gold mart builder (DuckDB) and baseline predictor.
src/reporting/	SVG visualization generation.
sql/flight_analysis.sql	Analytical SQL queries (DuckDB-oriented).
data/raw/, bronze/, silver/, gold/	Lakehouse data layers (large files often gitignored).
docs/	Architecture, data model, data dictionary, team handbook, data product guide.
tests/	unittest suite; includes fixtures/ for hermetic CI.
reports/	Validation reports, figures/, project reports/PDF artifacts.
dashboards/	Streamlit app.
models/	ML model/metrics outputs (when generated).
.github/workflows/ci.yml	CI pipeline configuration.
requirements.txt / requirements-dev.txt	Runtime and dev dependencies.
run_pipeline.py	Local sequential runner (non-Airflow).
README.md	Project overview and run instructions.

Application entry points

- Pipeline (Airflow-style runner script): `python airflow/dags/flight_delay_pipeline.py` (supports `--task` and `--simulate-failure`).
- Sequential local runner: `python run_pipeline.py`.
- Stage CLIs: `src/ingestion/ingest.py`, `src/transformations/clean_flights.py`, `src/validation/validate_flights.py`, `src/analytics/build_gold.py`, `src/reporting/visualize.py`, `src/analytics/predict_delay.py`.
- Dashboard: `streamlit run dashboards/streamlit_app.py`.

Where main business logic lives

- Transformation/feature logic: `clean_flights.py` (`_combine_datetime`, `clean_data`, `largest_delay_factor`/status/severity derivation).
- Quality rules/gate: `validate_flights.py` (custom validator, GX framework suite integration, report generation, `validate()` exit-on-fail).
- Analytical modeling: `build_gold.py` (DuckDB SQL aggregations and mart/dimension materialization).

Configuration, environment, dependency files

- `requirements.txt`, `requirements-dev.txt` (dependencies; CI uses dev).
- `.github/workflows/ci.yml` (CI env vars like `PYTHONPATH`, `USE_FIXTURES`, optional `RUN_SLOW_E2E`/`RUN_DATA_TESTS`).
- Airflow runtime config (`AIRFLOW_HOME`, metadata DB) is not fully committed/provisioned in repo; DAG is runnable via script/`dag.test()` patterns depending on environment.

5. Detailed Code Explanation (Key Files)

airflow/dags/flight_delay_pipeline.py

Defines `flight_delay_lakehouse_pipeline` (DAG) with tasks: ingest_bronze, clean_transform_silver, data_quality_validation, build_gold_marts, generate_visualizations. Uses `_run_stage` wrappers for start/success/error logging and `notify_failure` callback (prints/logs; optional webhook via `PIPELINE_ALERT_WEBHOOK`). Default args include retries and `retry_delay=5 minutes`. CLI supports `--task` and `--simulate-failure`. Note: the CLI `__main__` uses `dag.test()` (Airflow 2.5+ style) for full runs; task execution uses `task.execute({})`.

src/ingestion/ingest.py

`get_default_paths()` resolves raw/bronze defaults relative to project root. `ingest_data()` validates input exists, reads CSV with pandas (string dtypes for key ID/code columns), writes Parquet via PyArrow/Snappy, logs row/column counts, sizes, compression ratio, and elapsed time. Returns metrics dict.

src/transformations/clean_flights.py

`COLUMN_MAPPING` standardizes BTS columns to snake_case. `_combine_datetime()` parses YEAR/MONTH/DAY + HHMM with zero-padding and handles midnight rollover (`2400` -> next day 00:00). `clean_data()` builds datetime columns, derives `route`, `departure_date/hour/day_of_week`, delay aliases/duration, `flight_status`, surrogate `flight_id`, `largest_delay_factor` (priority/cause logic), `delay_severity`, and boolean cause flags; drops raw HHMM helper columns; writes Silver Parquet.

src/validation/validate_flights.py

Implements `CustomExpectationsValidator` producing GE-compatible result objects (table row count, column exists/not-null/unique/range/in-set, missingness thresholds). `run_custom_expectations_suite()` defines required columns and checks (including domain sets/ranges). `run_great_expectations_framework_suite()` attempts GX 1.x fluent/ephemeral context execution when installed and normalizes results; `run_great_expectations_suite()` combines GX-native + custom checks (or fallback) and records GX metadata. `generate_markdown_report()` renders a report; `validate()` writes JSON/MD and exits with code 1 on failure (quality gate). Supports `--simulate-failure`.

src/analytics/build_gold.py

Connects to DuckDB, creates a view over Silver Parquet, executes aggregation SQL for canonical marts (airline, route top corridors, delay factor, airport bottlenecks, temporal trends, aircraft/tail summary), writes each to `data/gold/*.parquet`, and materializes physical dimension tables `dim_airline.parquet` and `dim_airport.parquet` for star-schema alignment.

src/reporting/visualize.py

Reads Gold marts and generates hand-crafted SVG charts with `_svg_text()` XML escaping and dynamic canvas/footer heights (fixes overflow/invalid XML issues). Outputs: carrier comparison, worst routes, hourly trend, delay factor distribution, airport bottlenecks.

Tests and CI

`tests/test_*.py` use `unittest` to verify schemas, invariants, validation behavior/gate, Gold mart existence/invariants, and Airflow DAG structure/dependencies/failure hooks (Airflow-dependent tests skip when unavailable; heavy e2e gated). `.github/workflows/ci.yml` installs `requirements-dev.txt`, compiles key modules, and runs the unittest suite (fails on errors). `tests/fixtures/sample_flights.csv` supports hermetic runs where full raw data is unavailable.

Data flow through the code (example)

Example: A delayed flight row in raw CSV -> ingested to Bronze with original columns -> transformed in Silver: `AIRLINE` becomes `airline`, HHMM parsed to datetimes, `route` computed (`origin-destination`), `delay_minutes`/severity/factor derived, `flight_id` assigned -> validation checks ranges/domains/nulls; if valid, DuckDB groups by `airline`/`departure_hour`/`largest_delay_factor` to produce Gold marts -> visualization reads those marts and renders SVG bars/lines.

6. End-to-End Workflow (Runtime Behavior)

When the application starts

- Non-Airflow: `run_pipeline.py` prints stage banners and sequentially imports/calls `ingest_data()`, `clean_data()`, `validate()`, `build_gold()`, `generate_all_visualizations()`.
- Airflow DAG runner: `flight_delay_pipeline.py` parses CLI args; may override validation callable for `--simulate-failure`; executes one task or runs `dag.test()` depending on args.

When a user interacts

- Batch pipeline: interaction is mainly CLI/operator-driven (run script, run tests, view reports/figures).
- Streamlit: user runs `streamlit run dashboards/streamlit_app.py`; app loads Gold Parquet files and renders tables/charts interactively.

When a request is sent / data is processed

There is no HTTP request-response API in the core pipeline. "Requests" are function/task invocations: each stage reads input files (CSV/Parquet), processes in memory (pandas/DuckDB), and writes output artifacts.

Database access and response generation

- Primary storage access is file I/O to Parquet/CSV/JSON/MD/SVG. DuckDB acts as an embedded query engine (no separate DB server).
- "Responses" are generated artifacts: Parquet marts, validation reports, SVG figures, logs/stdout, and (optionally) ML model/metrics files.

7. Database and Data Flow

Database structure (actual implementation)

The project uses a file-based lakehouse, not a managed relational database server. Logical star-schema concepts are documented; physically, Silver is the fact-like Parquet table and Gold contains aggregate marts plus materialized dimension Parquet files (`dim_airline`, `dim_airport`).

Tables/collections/schemas/relationships (logical)

Entity	Physical form	Key/grain (implemented/logical)
Raw flights	`data/raw/flights.csv`	Source rows; BTS columns.
Bronze flights	`data/bronze/flights.parquet`	Source-preserving rows.
Silver fact	`data/silver/flights_cleaned.parquet`	1 row/scheduled operation; PK `flight_id`. Logical FKs: airline, origin, destination,
Gold marts	`data/gold/*.parquet`	Aggregates by airline, route, delay factor, airport, hour, tail number.
Dimensions (physical)	`dim_airline.parquet`, `dim_airport.parquet`	Materialized code lists for star-schema alignment.

CRUD operations

- Create/Write: ingestion and transformations write Parquet; validation writes JSON/MD; visualization writes SVG; Gold builder writes marts/dims.
- Read: each downstream stage reads upstream artifacts (CSV/Parquet).
- Update/Delete: implemented as overwrite/regeneration of output files (batch idempotent-style reruns), not row-level transactional CRUD.

How data moves between frontend, backend, and database

There is no traditional frontend-backend-database triplet. Data moves file-to-file through pipeline stages (backend processing). Streamlit (UI) reads Gold Parquet files directly from the filesystem. DuckDB reads Silver Parquet in-process and emits Gold files.

8. APIs and Integrations

Internal interfaces

- Python function interfaces between modules (e.g., `ingest_data`, `clean_data`, `validate`, `build_gold`, `generate_all_visualizations`).
- CLI interfaces via `argparse` in stage scripts and DAG runner.

External services/integrations

Integration	Purpose	Details
Apache Airflow	Orchestration runtime	DAG/tasks; execution depends on Airflow environment. `dag.test()`/task execution used in runn
Great Expectations	Data quality framework	Used when installed; falls back to custom checks with metadata. Dependency declared in requi
Optional webhook	Failure notification	`notify_failure` POSTs JSON to `PIPELINE_ALERT_WEBHOOK` if set; otherwise logs only.
GitHub Actions	CI automation	Workflow triggers on push/PR to main; installs deps, compiles, runs tests.
Kaggle/BTS dataset source	Input data origin	Raw CSV expected at `data/raw/flights.csv` (often gitignored due to size).

Authentication and authorization

No user authentication/authorization system is implemented in the pipeline or Streamlit dashboard (local capstone/data pipeline). Airflow security (RBAC/users) and webhook auth would be environment/deployment concerns, not implemented in-repo code.

9. Installation and Execution (Step-by-Step)

1. Set up development environment

```
cd "Flight Delay Lakehouse"
python -m venv .venv
source .venv/bin/activate   # Windows: .venv\Scripts\activate
python --version   # 3.10+ recommended
```

2. Install dependencies

```
pip install -r requirements-dev.txt
# For GX-native validation path, ensure great-expectations installs successfully
```

3. Configure environment variables (optional)

- `PIPELINE_ALERT_WEBHOOK` - optional URL for Airflow failure notifications.
- `USE_FIXTURES=1` - test/CI hint; fixtures exist at `tests/fixtures/sample_flights.csv`.
- `RUN_SLOW_E2E=1` / `RUN_DATA_TESTS=1` - enable heavier tests when artifacts/resources are available.
- Airflow service deployment would require `AIRFLOW_HOME` and metadata DB setup (not fully scripted in repo).

4. Provide input data

Place the raw dataset at `data/raw/flights.csv` (BTS/Kaggle On-Time Performance format, 31 expected columns). Large raw/derived Parquet files may be gitignored; ensure they exist locally for full-data runs.

5. Run the application locally

```
# Option A: sequential runner
python run_pipeline.py

# Option B: Airflow DAG runner script
python airflow/dags/flight_delay_pipeline.py

# Option C: single Airflow task
python airflow/dags/flight_delay_pipeline.py --task ingest_bronze
python airflow/dags/flight_delay_pipeline.py --task clean_transform_silver
python airflow/dags/flight_delay_pipeline.py --task data_quality_validation
python airflow/dags/flight_delay_pipeline.py --task build_gold_marts
python airflow/dags/flight_delay_pipeline.py --task generate_visualizations
```

6. Connect the database

No external database connection is required. DuckDB runs embedded and reads/writes local Parquet files.

7. Test everything works

```
python -m py_compile src/ingestion/ingest.py src/transformations/clean_flights.py
src/validation/validate_flights.py src/analytics/build_gold.py src/reporting/visualize.py
airflow/dags/flight_delay_pipeline.py run_pipeline.py
python -m unittest discover -s tests -p "test_*.py" -v
```

Expected in a properly provisioned environment: tests pass (some may skip based on Airflow availability/data/artifacts/slow-test flags). Validate outputs exist: `data/bronze|silver|gold`, `reports/validation_*`, `reports/figures/*.svg`.

10. Important Features

Feature	What it does	How it works / files

Bronze ingestion	Preserves raw data in efficient columnar format.	`src/ingestion/ingest.py`: pandas read_csv + PyArrow to_parquet(Snappy).
Silver cleaning & enrichment	Standardizes schema and derives analytical features.	`src/transformations/clean_flights.py`: renaming, HHMM/2400 datetime parsing, route/time fields,
Data quality gate	Prevents bad Silver data from feeding Gold.	`src/validation/validate_flights.py`: GX-native (if installed) + custom checks; JSON/MD reports; `sy
Gold OLAP marts	Produces business-ready aggregates.	`src/analytics/build_gold.py`: DuckDB SQL view over Silver -> marts + `dim_airline`/`dim_airport`.
SVG reporting	Creates visual summaries of Gold marts.	`src/reporting/visualize.py`: XML-safe, dynamic-height SVG charts.
Airflow orchestration	Automates stage order and reliability behavior.	`airflow/dags/flight_delay_pipeline.py`: DAG, retries/retry_delay, logging, failure callback, strict de
Dashboard (data product)	Interactive exploration of marts.	`dashboards/streamlit_app.py`: Streamlit reads Gold Parquet.
Baseline ML (data product)	Educational delay prediction baseline.	`src/analytics/predict_delay.py`: sampled RandomForest, metrics/model saved to `models/`.

11. Security and Error Handling

Authentication and authorization

Not implemented in application code (local batch pipeline/dashboard). No login, RBAC, or API auth layer. If deployed with Airflow services/UI, platform-level security must be configured separately.

Security mechanisms present

- Dependency/CI checks compile modules and run tests (`.github/workflows/ci.yml`).
- Data quality gate blocks downstream propagation of invalid data.
- Webhook notification is optional and uses environment variable configuration; failures in webhook posting are caught and logged.
- No secrets should be committed; `.gitignore` excludes large/local artifacts (and should be used for env secrets if added).

Validation and error handling

- Input existence checks (e.g., missing Bronze/Silver/raw files raise `FileNotFoundError`).
- Schema/column expectations in validation; missing columns produce failed checks/results.
- Transformation handles HHMM parsing with coercion/rollover logic; cancellation/diversion/status logic accounts for structural nulls.
- Airflow wrapper logs exceptions and re-raises to enforce dependency blocking; `notify_failure` records context.
- CLI validation uses non-zero exit on failure (usable in CI/orchestration).

Potential security/robustness weaknesses (observed)

- No authentication for Streamlit dashboard; not suitable for public exposure without controls.
- Webhook payload is unauthenticated/simple JSON; endpoint security depends on deployment.
- Pipeline assumes trusted local files and environment; limited input sanitization beyond expected schema/quality checks.
- Great Expectations native path availability depends on environment installation (fallback works, but governance posture differs by environment).
- Large datasets and generated artifacts require careful storage/permissions handling; repo/workspace may be on shared/external media depending on setup.

12. Current Project Status

Implemented (verified in current workspace)

- Medallion pipeline modules, Airflow DAG (5 tasks, retries/delay/logging/failure callback/blocking), validation (GX integration path + custom checks), Gold marts + `dim_airline`/`dim_airport`, SVG reporting fixes, tests, CI workflow, docs updates, fixtures, Streamlit dashboard, baseline ML predictor.
- Artifacts present: `data/bronze/silver/gold` Parquet outputs, `reports/validation_results.json|.md`, `reports/figures/*.svg`.
- Recent local test run: 24 tests total, 24 OK, 7 skipped (environment/optional gating).

Incomplete / environment-dependent

- Airflow scheduler/webserver/metadata DB deployment is not fully packaged in-repo (DAG is ready; platform setup may be required).
- GX-native execution in the observed local `.venv` may be unavailable if `great-expectations` is not installed there (fallback is explicit; CI/dev requirements declare it).
- Streamlit/ML dependencies may need to be installed in the active environment (declared in requirements).

Known issues / caveats

- Data-dependent tests can be sensitive to whether full raw/silver/gold artifacts exist; fixture/skip logic mitigates but may vary by environment.
- Validation row-count thresholds are tuned for the full dataset; tiny fixture datasets can fail the volume gate by design.

- Legacy/non-canonical Gold files may exist from earlier runs; docs identify canonical outputs.

Temporary/TODO-like items

No explicit TODO markers were identified as blocking core functionality in the main pipeline modules during this analysis; however, deployment hardening (Airflow infra, secrets/auth for dashboard/webhook) remains an operational improvement area.

13. Challenges and Technical Decisions

- Large dataset handling: Parquet + DuckDB chosen for local analytics efficiency instead of Spark/Kafka (appropriate for capstone scale/scope).
- Structural nulls: transformation/validation logic must distinguish legitimate missingness (cancelled/diverted/cause breakdown) from data defects.
- Time parsing edge case: aviation HHMM `2400` rollover handled to avoid NaT coercion.
- Validation portability: GX framework integration implemented with a custom GE-style fallback so the pipeline remains explainable and runnable across environments.
- Reporting robustness: SVGs generated as XML-safe text with dynamic layout to prevent viewer/parsing issues.
- Orchestration reliability: strict task dependencies + retries/delay + failure callback chosen to make failures visible and prevent partial downstream outputs.

14. Future Development

- Provision a reproducible Airflow runtime (Docker Compose/AIRFLOW_HOME/DB) and add DAG import/execution tests in CI where feasible.
- Parameterize validation thresholds/profiles (full dataset vs fixture/sample) for smoother local/CI runs.
- Add data contracts/schema versioning and lineage metadata (e.g., run IDs, source checksums).
- Harden dashboard (auth, deployment config) and expand BI (filters, drill-downs, KPIs).
- Improve ML baseline: feature pipeline, cross-validation, calibration, backtesting, and model registry; clarify it as optional/educational.
- Performance: incremental processing/partitioning if dataset grows or refresh cadence increases.
- Security: secrets management, webhook signing/auth, and environment isolation for deployed components.

15. Learning Roadmap (For a New Contributor)

Concepts to learn first

- Python data stack basics: pandas, file I/O, virtual environments.
- Data lakehouse concepts: Bronze/Silver/Gold (Medallion), Parquet/columnar storage.
- Data quality: expectations, validation gates, handling nulls/edge cases.
- SQL analytics with DuckDB (GROUP BY, CTEs, window functions).
- Workflow orchestration basics with Airflow (DAGs, operators, retries, dependencies).
- Testing/CI fundamentals: unittest, reproducible environments, GitHub Actions.

Files to study first (in order)

```
1) README.md
2) docs/architecture.md
3) docs/data_model.md and docs/data_dictionary.md
4) run_pipeline.py (simple end-to-end flow)
5) src/ingestion/ingest.py
6) src/transformations/clean_flights.py
7) src/validation/validate_flights.py
8) src/analytics/build_gold.py
9) src/reporting/visualize.py
10) airflow/dags/flight_delay_pipeline.py
```

```
11) tests/test_*.py and .github/workflows/ci.yml
```

Most important modules

- `clean_flights.py` (core business/feature logic) and `validate_flights.py` (trust/governance).
- `build_gold.py` (analytics value) and the Airflow DAG (operational reliability).

Practical exercises

- Run a single stage CLI and inspect its output artifact.
- Use `--simulate-failure` in validation/DAG and observe blocking behavior/logs.
- Add one safe validation expectation (e.g., a new range) and update reports/tests.
- Add a new Gold mart in DuckDB and a corresponding SVG chart.
- Run tests locally, then trace a failing test to the responsible module.

How to become independent

Start by rerunning the pipeline on existing artifacts, then modify one small, test-covered behavior at a time (feature derivation, validation rule, or Gold query). Keep docs (`docs/data_model.md`, `docs/data_dictionary.md`, README) synchronized with code changes, and always verify with unittest/CI.

Appendix A: Verified/Observed Facts (from this workspace)

- Silver schema observed at 44 columns; `flight_id` unique/non-null in current artifact.
- Canonical Gold marts + physical dims present: airline/route/delay_factor/airport_bottlenecks/temporal_trends/aircraft + dim_airline/dim_airport.
- Validation report (current): success=True, 50 evaluated/passed (100%); reports refreshed in `reports/`.
- SVG figures parse as valid XML after escaping/dynamic-height fixes.
- Airflow 2.9.3 declared; DAG structure verified in Airflow-enabled environment; full scheduler/webserver deployment not fully provisioned in-repo.

Note: Some metrics in repository docs/reports (e.g., compression ratios, historical run times, older "34/50 checks" text in legacy report) may reflect prior runs or different environments. This report prioritizes implementation-verified behavior and clearly marks environment-dependent items.